Heart Disease Prediction System (HeartGuard)

Installation and Run Guide

This guide explains, step by step, how to open and run this project on a new computer: installing the required software, creating the MySQL database and tables, configuring the backend, installing Python and Node.js dependencies, starting the backend server, starting the frontend web app, and logging in as a user and as an admin.

1. Project Overview

The system predicts the risk level (Low / Moderate / High) of heart disease using a machine learning model (Random Forest Classifier). A user enters 11 clinical features (age, sex, chest pain type, resting BP, cholesterol, etc.) and the model returns a prediction with a probability score.

The project has four parts:

Part	Technology	Folder	What it does
Frontend	React + Vite	`Frontend/`	Web app for users and admin dashboard
Backend	Python FastAPI	`Backend/`	REST API, runs the ML model, saves results
Database	MySQL	`Database/`	Stores users, admins and predictions
ML Training	Jupyter Notebook	`ML_Model_Training/`	Shows how the model was trained (not needed to run the app)

How the parts are connected:

```
Browser (React Web App) ---> FastAPI backend (port 8000) ---> MySQL (port 3306)
                                        |
                                        +---> ML model (heart_disease_pipeline.jo
```

- The frontend talks to the backend using the address `http://localhost:8000`. This is already set in `Frontend/src/api/axios.js` (line 4), so nothing needs to be changed for the web version.
- The backend talks to the database using the connection details in the file `Backend/.env` (you create/edit this file in Step 3).

2. Folder Structure

```
HEART GURAD FYP/
|-- requirements.txt                    <- list of all Python libraries
|-- SETUP_GUIDE.html                    <- this setup guide
|
|-- Backend/
|   |-- .env                            <- database & secret key settings
|   |-- requirements.txt                <- Python library list (same as root)
|   |-- heart_disease_pipeline.joblib   <- trained ML model file
|   |-- app/
|       |-- main.py                     <- MAIN backend file (server starts here
|       |-- config.py                   <- reads settings from .env
|       |-- database.py                 <- database connection setup
|       |-- dependencies.py             <- database session helper
|       |-- utils_auth.py              <- JWT token verification
|       |-- core/
|       |   |-- security.py            <- password hashing & JWT creation
|       |   |-- model_loader.py        <- loads the ML model into memory
|       |-- models/
|       |   |-- user.py                <- User database table
|       |   |-- prediction.py          <- HeartDiseasePrediction table
|       |-- schemas/
|       |   |-- auth.py                <- register/login request schemas
|       |   |-- user.py                <- user response schema
|       |   |-- prediction.py          <- prediction input/output schemas
|       |-- routers/
|           |-- auth.py                <- /auth/register, /auth/login
|           |-- users.py               <- /users/me, /users/my-predictions
|           |-- predictions.py         <- /predictions/ (make prediction)
|           |-- admin.py               <- /admin/* (dashboard, users, etc.)
|
|-- Frontend/
|   |-- package.json                    <- list of Node.js packages
|   |-- vite.config.js                  <- Vite build configuration
|   |-- index.html                      <- HTML entry point
|   |-- src/
|       |-- main.jsx                    <- app starting point
|       |-- App.jsx                     <- routes and navigation
|       |-- api/
```

```
|       |   |-- axios.js                <- backend address (URL) is set here
|       |-- context/
|       |   |-- AuthContext.jsx         <- login/logout state management
|       |-- components/                 <- reusable UI components (Navbar, etc.)
|       |-- pages/                      <- all app screens
|           |-- LandingPage.jsx         <- public homepage
|           |-- Login.jsx               <- login form
|           |-- Register.jsx            <- registration form
|           |-- user/                   <- user screens (Dashboard, Predict, His
|           |-- admin/                  <- admin screens (Dashboard, Users, Pred
|
|-- Database/
|   |-- schema.sql                      <- SQL script to create the database & t
|
|-- ML_Model_Training/
    |-- ml_model.ipynb                  <- model training notebook (main)
    |-- model.ipynb                     <- alternative training notebook
    |-- heart_disease_datasets.csv      <- training dataset
    |-- heart_disease_pipeline.joblib   <- exported trained model
    |-- heart_disease_model.pkl         <- alternative exported model
```

3. Software Requirements

Install the following software before starting:

Software	Version	Download link	Used for
Python	3.11	https://www.python.org/downloads/	Backend
Node.js	18 or newer (LTS)	https://nodejs.org/	Frontend
MySQL Server + MySQL Workbench	8.0 or newer	https://dev.mysql.com/downloads/installer/	Database
Google Chrome or Microsoft Edge	Latest	—	To open the web app
VS Code (optional)	Latest	https://code.visualstudio.com/	To open and view the code

Installation notes:

- **Python (Windows):** on the first screen of the installer, tick **"Add python.exe to PATH"**. Select Python 3.11 specifically.
- **Node.js:** download the LTS version. The installer will automatically add `node` and `npm` to your PATH.
- **MySQL:** during installation, set a root password and remember it (example: `admin`). Keep the port as **3306**. MySQL Workbench is included in the MySQL Installer for Windows.
- An internet connection is needed the first time, to download the Python and Node.js packages.

Check that everything is installed (open a new Command Prompt / terminal):

```
python --version
node --version
npm --version
```

Note: On macOS / Linux, use `python3` instead of `python` in all the commands in this guide.

4. Step-by-Step Setup (First Time)

Step 1 — Copy the Project to the Computer

Copy the whole project folder from the pen drive to the computer, for example to `C:\FYP\` or the Desktop. Run it from the computer, not directly from the pen drive.

How to open a terminal inside a folder (Windows): open the folder in File Explorer, click the address bar, type `cmd` and press Enter. (In VS Code: right-click the folder and choose Open in Integrated Terminal.)

Step 2 — Create the MySQL Database and Tables

The project uses a MySQL database named `heart_disease`. The database is not inside the project folder. Each computer has its own MySQL, so the database must be created once on the new computer.

Option A — MySQL Workbench (easiest)

1. Open **MySQL Workbench 8.0 CE** and click the Local instance MySQL connection. Enter your MySQL root password.
2. Go to **File → Open SQL Script** and select the file:
   `Database/schema.sql`
3. Click the ⚡ **Execute** button (or press `Ctrl+Shift+Enter`).
4. Click the refresh icon in the Schemas panel. You should now see the `heart_disease` database with two tables: `users` and `heart_predictions`.

Option B — Command line

```
mysql -u root -p
```

Enter your MySQL password, then type:

```
SOURCE C:/path/to/HEART GURAD FYP/Database/schema.sql;
EXIT;
```

(Replace `C:/path/to/` with the actual path where you copied the project.)

The `schema.sql` script creates the database and both tables automatically. You can verify by running `SHOW TABLES;` in the `heart_disease` database.

Step 3 — Connect the Backend to the Database (.env file)

The backend reads the database username, password, and other settings from a file named `.env` inside the `Backend` folder.

1. Open the `Backend` folder.
2. Open the `.env` file with Notepad or VS Code. If the file does not exist, create a new file named exactly `.env` (not `.env.txt`).
3. Edit the file contents to match your MySQL credentials:

```
DATABASE_URL = mysql+mysqlconnector://root:admin@localhost/heart_disease
SECRET_KEY = admin
ALGORITHM = HS256
ACCESS_TOKEN_EXPIRE_MINUTES = 60
MODEL_PATH = heart_disease_pipeline.joblib
```

   – The format is:
   `mysql+mysqlconnector://USERNAME:PASSWORD@localhost/heart_disease`
   – Replace `root` with your MySQL username and `admin` (after the colon) with your MySQL password.
   – `SECRET_KEY` can be any long random text. It is used to create login tokens.
   – If your password contains special characters like `@`, `#`, `/` or `:`, replace them with their URL codes (for example `@` becomes `%40`).

4. Save the file.

Important: The file name must be exactly `.env`, not `.env.txt`. In File Explorer, turn on **View > Show > File name extensions** to check.

Step 4 — Install the Backend (Python) Libraries

Open a terminal inside the `Backend` folder and run these commands one by one:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

- `python -m venv venv` creates a virtual environment (a private folder for this project's libraries). This is done only once.
- `venv\Scripts\activate` activates it. After this, `(venv)` appears at the start of the terminal line.

– macOS / Linux: `source venv/bin/activate`

– Windows PowerShell: `venv\Scripts\Activate.ps1`

- `pip install -r requirements.txt` installs all libraries listed in `requirements.txt` (FastAPI, Uvicorn, SQLAlchemy, MySQL connector, pandas, scikit-learn, bcrypt, etc.).

Step 5 — Start the Backend Server

In the same terminal (with `(venv)` active), inside the `Backend` folder, run:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

The backend is running when you see:

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```

At this moment the backend has also connected to MySQL. The tables were already created by the `schema.sql` script in Step 2. (SQLAlchemy's `create_all` will also create them automatically if they don't exist.)

Check that it works by opening these addresses in the browser:

- `http://127.0.0.1:8000` — shows `{"message": "Heart Disease Prediction System API is running."}`
- `http://127.0.0.1:8000/docs` — interactive API page (Swagger UI) where every API can be tested with the Try it out button

Keep this terminal open. Closing it stops the backend. Use a new terminal for the next step.

Step 6 — Install and Run the Frontend Web App

Open a **new terminal** inside the `Frontend` folder and run:

```
npm install
npm run dev
```

- `npm install` downloads all JavaScript packages listed in `package.json` (React, Axios, React Router, Vite, etc.). This is needed only the first time.
- `npm run dev` starts the Vite development server and opens the web app.

When you see the following output, the frontend is running:

```
  VITE v8.x.x  ready in xxx ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

Open `http://localhost:5173` in Google Chrome or Microsoft Edge. You should see the HeartGuard landing page.

Connection between frontend and backend: The web app sends all requests to `http://localhost:8000`. This is already set in `Frontend/src/api/axios.js` (line 4), so no change is needed as long as the backend from Step 5 is running on the same computer.

Step 7 — Create the Admin Account (First Time Only)

A new database is empty, so there is no admin account yet. Create one from the app:

1. On the web app's landing page, click **Get Started** or go to the Register page.
2. Fill in the registration form:
   – **Full Name:** System Admin
   – **Username:** admin
   – **Email:** admin@heartguard.com
   – **Password:** (choose a password, minimum 5 characters)
   – **Role:** select **Admin**
3. Click Register. You should see a success message.
4. Go to the Login page and log in with the admin username and password.
5. You will be redirected to the **Admin Dashboard** where you can see all users, all predictions, and dashboard statistics.

To create a regular user account, repeat the steps above but select **User** as the role. Regular users can:

- Make heart disease predictions
- View their prediction history
- View their profile

5. Login Details

Role	Username	Password	Notes
Admin	`admin`	`Admin@123`	Create it first using the Register page (Step 7)
User	Any username you register	Any password	Register on the Register page with role "User"

From now on, log in with the Login page using the same username and password. (Any name, email and password can be used; the ones above are just suggestions.)

Note: Passwords are stored in the database as secure bcrypt hashes, not as plain text.

6. How to Use and Test the System

When the web app opens, the landing page shows two options: the User portal and the Admin portal.

6.1 Test as a User

1. On the landing page, click **Get Started** to go to the Login page.
2. Log in with a user account. (Or go to the Register page first to create one with role "User".)
3. On the **Dashboard**, you will see a welcome message and quick links.
4. Click **Predict** (or go to the Predict page from the Navbar). Fill in the form with the 11 clinical features:
   – **Age:** patient's age (1–120)
   – **Sex:** M or F
   – **Chest Pain Type:** ATA, NAP, ASY, or TA
   – **Resting BP:** resting blood pressure in mmHg (50–250)
   – **Cholesterol:** serum cholesterol in mg/dl (0–700)
   – **Fasting BS:** 1 if fasting blood sugar > 120 mg/dl, else 0
   – **Resting ECG:** Normal, ST, or LVH
   – **Max HR:** maximum heart rate achieved (60–250)
   – **Exercise Angina:** Y or N
   – **Oldpeak:** ST depression value (-10 to 10)
   – **ST Slope:** Up, Flat, or Down
5. Click **Submit**. The result screen shows:
   – The prediction: **Heart Disease** or **Normal**
   – The probability (%) of heart disease
   – The risk level: **Low**, **Moderate**, or **High**
6. Go to **My Predictions** from the Navbar to see all past predictions with timestamps.

6.2 Test as Admin (Dashboard)

1. Go to the landing page and click **Get Started**.
2. Log in with `admin` / `Admin@123` (created in Step 7).

3. You will be redirected to the **Admin Dashboard**. Use the Navbar to access:
   – **Dashboard** — overview with total users, total predictions, and high-risk prediction count
   – **All Users** — list of all registered users with their details
   – **All Predictions** — list of all predictions made by all users
   – **User Predictions** — view predictions for a specific user (click from the All Users page)

6.3 Check the Saved Data in MySQL

In MySQL Workbench, run:

```
USE heart_disease;
SELECT * FROM users;
SELECT * FROM heart_predictions;
```

Every prediction made in the app is saved in these tables.

7. Quick Start (Every Time After the First Setup)

Make sure the MySQL service is running (on Windows it starts automatically with the computer). Then:

Terminal 1 — Backend

```
cd Backend
venv\Scripts\activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Terminal 2 — Frontend

```
cd Frontend
npm run dev
```

Then open `http://localhost:5173` in Google Chrome or Microsoft Edge and log in (e.g. as admin with `admin` / `Admin@123`).

To stop: press `Ctrl+C` in each terminal window.

8. Optional — Use the Developer's Existing Data

By default, the new computer starts with an empty database. If you want the examiner to see the same users, admin account and predictions as on the developer's laptop, export the database and import it on the new computer.

On the developer's laptop (export):

1. Open MySQL Workbench and go to **Server > Data Export**.
2. Tick `heart_disease`, choose **Export to Self-Contained File**, and save it as `heart_disease_backup.sql`.
3. Click **Start Export** and copy the `.sql` file to the pen drive.

On the new computer (import):

1. Do Step 2 (create the empty `heart_disease` database using `schema.sql`).
2. In MySQL Workbench, go to **Server > Data Import**.
3. Choose **Import from Self-Contained File** and select `heart_disease_backup.sql`.
4. Set **Default Target Schema** to `heart_disease` and click **Start Import**.

After importing, the developer's existing admin email and password will work, and Step 7 is not needed.

(Command-line alternative: export with `mysqldump -u root -p heart_disease > heart_disease_backup.sql` and import with `mysql -u root -p heart_disease < heart_disease_backup.sql`.)

9. Optional — Build the Frontend for Production

The backend and frontend can run in development mode (as shown in Step 5 and Step 6). If you want to create an optimized production build of the frontend:

1. Open a terminal in the `Frontend` directory and run:

   ```
   npm run build
   ```

   This compiles and minifies the React application into static HTML, JavaScript, and CSS files inside the `Frontend/dist` folder.
2. To test and preview the production build locally without Vite's dev server, run:

```
npm run preview
```

This starts a local preview server at `http://localhost:4173`.

3. Open `http://localhost:4173` in Chrome to access the production build.

10. Backend API Endpoints

Method	Endpoint	Purpose
GET	`/`	Health check
POST	`/auth/register`	User registration (User or Admin)
POST	`/auth/login`	User login and JWT access token
GET	`/users/me`	Current logged-in user profile
GET	`/users/my-predictions`	Current user's prediction history
POST	`/predictions/`	Heart disease risk prediction
GET	`/admin/dashboard-stats`	Admin dashboard statistics
GET	`/admin/users`	List of all registered users
GET	`/admin/predictions`	List of all predictions in the system
GET	`/admin/users/{id}/predictions`	Predictions made by a specific user

Full details and an interactive **Try it out** button for each endpoint: `http://127.0.0.1:8000/docs`

11. Troubleshooting

Problem	Solution
`RuntimeError: DATABASE_URL is not set`	The `.env` file is missing or has the wrong name. Repeat Step 3. The name must be `.env`, not `.env.txt`.
`Access denied for user 'root'@'localhost'`	Wrong MySQL username or password in `.env`. Fix it and restart the backend.
`Unknown database 'heart_disease'`	The database was not created. Do Step 2 using MySQL Workbench and `schema.sql`.
`Can't connect to MySQL server on 'localhost:3306'`	MySQL is not running. Press Win+R, type `services.msc`, find MySQL80 (or MySQL) and click Start.
Heart prediction error / `scikit-learn` unpickling error	Wrong scikit-learn version. Run `pip install -r requirements.txt` (with venv active) and restart the backend.
`'python' is not recognized`	Python is not in PATH. Reinstall Python 3.11 and tick "Add python.exe to PATH", or use `py` instead of `python`.
`'uvicorn' is not recognized`	The venv is not active (no `(venv)` in the terminal). Run `venv\Scripts\activate` first, or use `python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000`.
`You must pass the application as an import string to enable 'reload'`	This appears when starting with `python app/main.py`. Start the backend with the `uvicorn app.main:app --reload ...` command from Step 5 instead.
PowerShell: running scripts is disabled on this system	Use Command Prompt (`cmd`) instead, or run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.
A venv folder copied from another computer does not work	Delete the `venv` folder and repeat Step 4.
`Address already in use / error 10048 on port 8000`	Another backend is already running. Close the other terminal (or restart the computer) and start again.

`'node' is not recognized` or `'npm' is not recognized`	Node.js is not in PATH. Reinstall Node.js (LTS version) and open a new terminal.
Web app shows a network error or "Failed to fetch"	The backend is not running. Start it (Step 5) and check `http://127.0.0.1:8000` .
Port 5173 already in use	Vite will automatically try the next available port (5174). Check the terminal output for the exact URL.
Admin login: "Incorrect username or password"	The admin account does not exist in this computer's database. Create it with the Register page (Step 7).

Note: The Jupyter notebooks in `ML_Model_Training/` were used during development for exploratory data analysis, evaluation, and model training. To run the project, always start the backend with `uvicorn app.main:app ...` as shown in Step 5. The pre-trained pipeline (`Backend/heart_disease_pipeline.joblib`) is already included and loaded automatically on startup.

12. Technologies Used

- **Frontend:** React 19, Vite, Axios, React Router DOM, React Icons
- **Backend:** FastAPI, Uvicorn, SQLAlchemy, Pydantic, bcrypt, python-jose (JWT tokens)
- **Database:** MySQL 8.0 (mysql-connector-python)
- **Machine Learning:** scikit-learn (Random Forest Classifier), pandas, NumPy, joblib
- **Model Training:** Jupyter Notebook (`ML_Model_Training/ml_model.ipynb` , `ML_Model_Training/model.ipynb`)
- **Data Visualization (Training):** Matplotlib, Seaborn

Disclaimer: This system is an academic project. The predictions are for educational purposes only and are not a medical diagnosis.